

run_d2co.py (Q&A Documentation)

Q: What does this run script control? **A:** It controls the full experiment loop: args, data loading, model instantiation, training with a specific loss/optimizer, and evaluation with shared metrics.

Q: Why is the model not responsible for training/evaluation? **A:** Keeping training logic in the script makes it easy to compare models under the same protocol and avoids mixing I/O/metrics with differentiable code.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
def get_args(...) # function
def get_loaders(...) # function
def get_gmm_mean(...) # function
def get_gmm_label(...) # function
def get_real_value(...) # function
def mae_rescale_to_second(...) # function
def test(...) # function
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataLoader
from sklearn.mixture import GaussianMixture
from collections import Counter
from model import D2Q, WideAndDeep
from utils import eval_mae, eval_xauc, eval_kl
import pandas as pd
import pickle as pkl
from torch.distributions import Normal
```

```
def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuairc')
```

```

parser.add_argument('--dataset_path', default='./dataset/')
parser.add_argument('--device', default='cuda:0')
parser.add_argument('--bsz', type=int, default=2048)
parser.add_argument('--log_interval', type=int, default=10)

parser.add_argument('--epoch', type=int, default=10)
parser.add_argument('--lr', type=float, default=0.1)
parser.add_argument('--weight_decay', type=float, default=1e-6)
parser.add_argument('--seed', type=int, default=42)

args = parser.parse_args()
return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataloader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def get_gmm_mean(df_train, n_bins):
    # using GMM to calculate the mean values of each distribution
    gmmMeanList = []
    durationBucketList = []
    playTimeList = []
    for _, (features, label) in enumerate(df_train):
        durationBucketList.extend(features['duration_bucket'].cpu().numpy().flatten())
        playTimeList.extend(label.cpu().numpy().flatten())
    durationBucketList = np.array(durationBucketList)
    playTimeList = np.array(playTimeList)

```

Q: What is get_args and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for get_args? **A:** Excerpt from the file:

```

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

```

Q: What is get_loaders and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_loaders`? **A:** Excerpt from the file:

```
def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataloader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders
```

Q: What is `get_gmm_mean` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_gmm_mean`? **A:** Excerpt from the file:

```
def get_gmm_mean(df_train, n_bins):
    # using GMM to calculate the mean values of each distribution
    gmmMeanList = []
    durationBucketList = []
    playTimeList = []
    for _, (features, label) in enumerate(df_train):
        durationBucketList.extend(features['duration_bucket'].cpu().numpy().flatten())
        playTimeList.extend(label.cpu().numpy().flatten())
    durationBucketList = np.array(durationBucketList)
    playTimeList = np.array(playTimeList)

    for d in range(n_bins):
        playTimeInBucket = playTimeList[durationBucketList == d].reshape(-1, 1)
        gm = GaussianMixture(n_components=2, init_params='kmeans', covariance_type='spherical', max_iter=500, random_state=61).fit(playTimeInBucket)
        means = np.sort(gm.means_.T[0])
        gmmMeanList.append([d, means[0], means[1]])

    # calculate num of row in each bucket
    numInEachBucket = Counter(durationBucketList)
    numInEachBucket = dict(sorted(numInEachBucket.items()))
    numInEachBucket = list(numInEachBucket.values())

    def freq_moving_ave(ls_v, ls_w, windows_size=5):
        ls_mul = np.array(ls_v) * np.array(ls_w)
        amount = pd.Series(ls_mul)
        amount_sum = amount.rolling(2*windows_size-1, min_periods=1, center=True).agg(lambda x: np.sum(x))

        weight = pd.Series(ls_w)
        weight_sum = weight.rolling(2*windows_size-1, min_periods=1, center=True).agg(lambda x: np.sum(x))

        return amount_sum/weight_sum

    # smoothing by frequent moving average
    gmmMeanList = np.array(gmmMeanList)
    nega_GMM_mean = dict(zip(gmmMeanList[:,0], freq_moving_ave(gmmMeanList[:,1], numInEachBucket, windows_size=5)))
    posi_GMM_mean = dict(zip(gmmMeanList[:,0], freq_moving_ave(gmmMeanList[:,2], numInEachBucket, windows_size=5)))
    return nega_GMM_mean, posi_GMM_mean
```

Q: What is `get_gmm_label` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_gmm_label`? **A:** Excerpt from the file:

```
def get_gmm_label(label, idx, nega_GMM_mean, posi_GMM_mean, alpha=1.0):
    p = nega_GMM_mean[idx]
    q = posi_GMM_mean[idx]
    gmm_label = (np.exp(alpha * label) - np.exp(alpha * q)) / (np.exp(alpha * p) - np.exp(alpha * q))
    return np.clip(gmm_label, 0, 1)
```

Q: What is `get_real_value` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_real_value`? **A:** Excerpt from the file:

```
def get_real_value(y, idx, nega_GMM_mean, posi_GMM_mean, alpha=1.0):
    p = nega_GMM_mean[idx]
    q = posi_GMM_mean[idx]
    real_y = np.log(y * (np.exp(alpha * p) - np.exp(alpha * q)) + np.exp(alpha * q)) / alpha
    return real_y
```

Q: What is `mae_rescale_to_second` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `mae_rescale_to_second`? **A:** Excerpt from the file:

```
def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuaiREC':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae * (6000 - 31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))
```

Q: What is `test` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `test`? **A:** Excerpt from the file:

```
def test(args, model, dataloaders, nega_GMM_mean, posi_GMM_mean):
    model.eval()
    labels, scores, predicts, durs = list(), list(), list(), list()
    with torch.no_grad():
        for _, (features, label) in enumerate(dataloaders['test']):
            bucket_index = features['duration_bucket']
            y = model(features)
            mapped_y, mapped_label = [], []
            for idx, gmm_y in zip(bucket_index, y.tolist()):
                mapped_y_value = get_real_value(gmm_y, idx.item(), nega_GMM_mean, posi_GMM_mean)
                mapped_y.append(mapped_y_value)
            labels.extend(label.tolist())
            scores.extend(mapped_y)
            durs.extend(features['duration'].squeeze().tolist())
    labels, scores = np.array(labels), np.array(scores)
    mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
    mae = mae_rescale_to_second(args.dataset_name, mae)
    print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))
```

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```
python run_d2co.py --dataset_name kuaiREC --dataset_path ./dataset --device cuda:0
# Internally: load dataloaders → build model → train → evaluate
```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```
import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataLoader
from sklearn.mixture import GaussianMixture
from collections import Counter
from model import D2Q, WideAndDeep
from utils import eval_mae, eval_xauc, eval_kl
import pandas as pd
import pickle as pkl
from torch.distributions import Normal

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataLoader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def get_gmm_mean(df_train, n_bins):
    # using GMM to calculate the mean values of each distribution
    gmmMeanList = []
    durationBucketList = []
    playTimeList = []
    for _, (features, label) in enumerate(df_train):
        durationBucketList.extend(features['duration_bucket'].cpu().numpy().flatten())
        playTimeList.extend(label.cpu().numpy().flatten())
    durationBucketList = np.array(durationBucketList)
    playTimeList = np.array(playTimeList)

    for d in range(n_bins):
        playTimeInBucket = playTimeList[durationBucketList == d].reshape(-1, 1)
        gm = GaussianMixture(n_components=2, init_params='kmeans', covariance_type='spherical', max_iter=500, random_state=61).fit(playTimeInBucket)
        means = np.sort(gm.means_.T[0])
        gmmMeanList.append([d, means[0], means[1]])

    # calculate num of row in each bucket
    numInEachBucket = Counter(durationBucketList)
    numInEachBucket = dict(sorted(numInEachBucket.items()))
    numInEachBucket = list(numInEachBucket.values())

def freq_moving_ave(ls_v, ls_w, windows_size=5):
    ls_mul = np.array(ls_v) * np.array(ls_w)
    amount = pd.Series(ls_mul)
    amount_sum = amount.rolling(2*windows_size-1, min_periods=1, center=True).agg(lambda x: np.sum(x))
```

```

weight = pd.Series(ls_w)
weight_sum = weight.rolling(2*windows_size-1, min_periods=1, center=True).agg(lambda x: np.sum(x))

return amount_sum/weight_sum

# smoothing by frequent moving average
gmmMeanList = np.array(gmmMeanList)
nega_GMM_mean = dict(zip(gmmMeanList[:,0],freq_moving_ave(gmmMeanList[:,1], numInEachBucket, windows_size=5)))
posi_GMM_mean = dict(zip(gmmMeanList[:,0],freq_moving_ave(gmmMeanList[:,2], numInEachBucket, windows_size=5)))
return nega_GMM_mean, posi_GMM_mean

def get_gmm_label(label, idx, nega_GMM_mean, posi_GMM_mean, alpha=1.0):
    p = nega_GMM_mean[idx]
    q = posi_GMM_mean[idx]
    gmm_label = (np.exp(alpha * label) - np.exp(alpha * q)) / (np.exp(alpha * p) - np.exp(alpha * q))
    return np.clip(gmm_label, 0, 1)

def get_real_value(y, idx, nega_GMM_mean, posi_GMM_mean, alpha=1.0):
    p = nega_GMM_mean[idx]
    q = posi_GMM_mean[idx]
    real_y = np.log(y * (np.exp(alpha * p) - np.exp(alpha * q)) + np.exp(alpha * q)) / alpha
    return real_y

def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuaiREC':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae * (6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))

def test(args, model, dataloaders, nega_GMM_mean, posi_GMM_mean):
    model.eval()
    labels, scores, predicts, durs = list(), list(), list(), list()
    with torch.no_grad():
        for _, (features, label) in enumerate(dataloaders['test']):
            bucket_index = features['duration_bucket']
            y = model(features)
            mapped_y, mapped_label = [], []
            for idx, gmm_y in zip(bucket_index, y.tolist()):
                mapped_y_value = get_real_value(gmm_y, idx.item(), nega_GMM_mean, posi_GMM_mean)
                mapped_y.append(mapped_y_value)
            labels.extend(label.tolist())
            scores.extend(mapped_y)
            durs.extend(features['duration'].squeeze().tolist())
    labels, scores = np.array(labels), np.array(scores)
    mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
    mae = mae_rescale_to_second(args.dataset_name, mae)
    print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))

if __name__ == '__main__':
    args = get_args()
    if args.seed > -1:
        np.random.seed(args.seed)
        torch.manual_seed(args.seed)
        torch.cuda.manual_seed(args.seed)
    res = {}
    torch.cuda.empty_cache()

    device = torch.device(args.device)

    dataloaders = get_loaders(args.dataset_name, args.dataset_path, device, args.bsz)
    model = WideAndDeep(dataloaders.description, embed_dim=16, mlp_dims=(512, 256, 128, 64, 32), dropout=0.0)
    model = model.to(device)

```

```

# train
dataloader_train = dataloaders['train']
bin_nums = 50
if args.dataset_name == "wechat":
    bin_nums = 17
nega_GMM_mean, posi_GMM_mean = get_gmm_mean(dataloader_train, bin_nums)

model.train()
criterion = torch.nn.MSELoss()
optimizer = torch.optim.Adagrad(params=model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
for epoch_i in range(1, args.epoch + 1):
    model.train()
    epoch_loss = 0.0
    total_loss = 0
    total_iters = len(dataloader_train)
    for i, (features, label) in enumerate(dataloader_train):
        y = model(features)
        bucket_index = features['duration_bucket']
        mapped_label = []
        for idx, label_origin in zip(bucket_index, label.tolist()):
            mapped_label_value = get_gmm_label(label_origin, idx.item(), nega_GMM_mean, posi_GMM_mean)
            mapped_label.append(mapped_label_value)
        mapped_label = torch.tensor(mapped_label, device=device)
        loss = criterion(y, mapped_label.float())
        model.zero_grad()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
        total_loss += loss.item()
        if (i + 1) % 10 == 0:
            print("    Iter {}/{} loss: {:.7f}".format(i + 1, total_iters + 1, total_loss/args.log_interval), end='\r')
            total_loss = 0
    print("Epoch {}/{} average Loss: {:.7f}".format(epoch_i, args.epoch, epoch_loss/total_iters))
test(args, model, dataloaders, nega_GMM_mean, posi_GMM_mean)

```